Chương 3
TIẾN TRÌNH

NỘI DUNG

1. **Khái niệm tiến trình**
2. **Định thời tiến trình**
3. **Các thao tác trên tiến trình**
4. **Truyền thông liên tiến trình (Interprocess Communication - IPC)**

MỤC TIÊU

- ✓ Giới thiệu các khái niệm về tiến trình
- ✓ Mô tả các đặc trưng khác nhau của tiến trình
- ✓ Khảo sát việc truyền thông liên tiến trình sử dụng cơ chế **shared memory** và cơ chế **message passing**
- ✓ Mô tả truyền thông trong hệ thống Client-Server

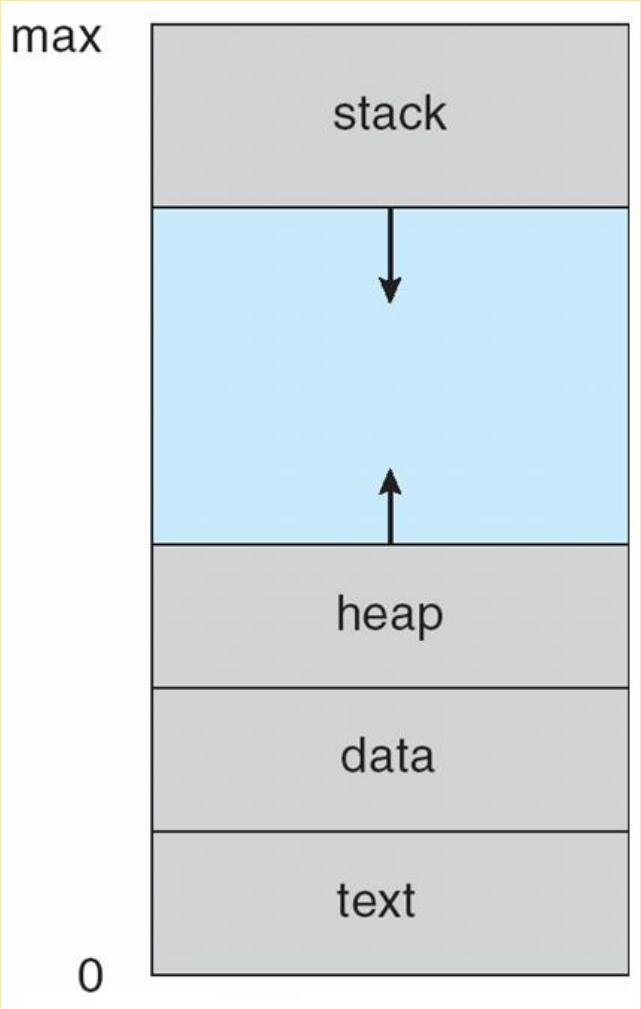
1. Khái niệm tiến trình

- HĐH thực thi nhiều loại chương trình khác nhau
  - Hệ thống bó (Batch System) – công việc (jobs)
  - Hệ thống chia sẻ thời gian (Time-shared System) – chương trình người dùng (user programs) hoặc tác vụ (tasks)
- Tiến trình – một chương trình đang thực thi
- Tiến trình gồm nhiều thành phần:
  - Mã chương trình (program code), còn gọi là phần văn bản (text)
  - Các hoạt động hiện hành: bộ đếm chương trình (program counter), thanh ghi (register)
  - Ngăn xếp (Stack): chứa đựng dữ liệu tạm
    - Các tham số, địa chỉ, biến cục bộ
  - Thành phần dữ liệu (Data section): chứa đựng biến toàn cục
  - Khối xếp (Heap): chứa bộ nhớ cấp phát động trong suốt thời gian chạy

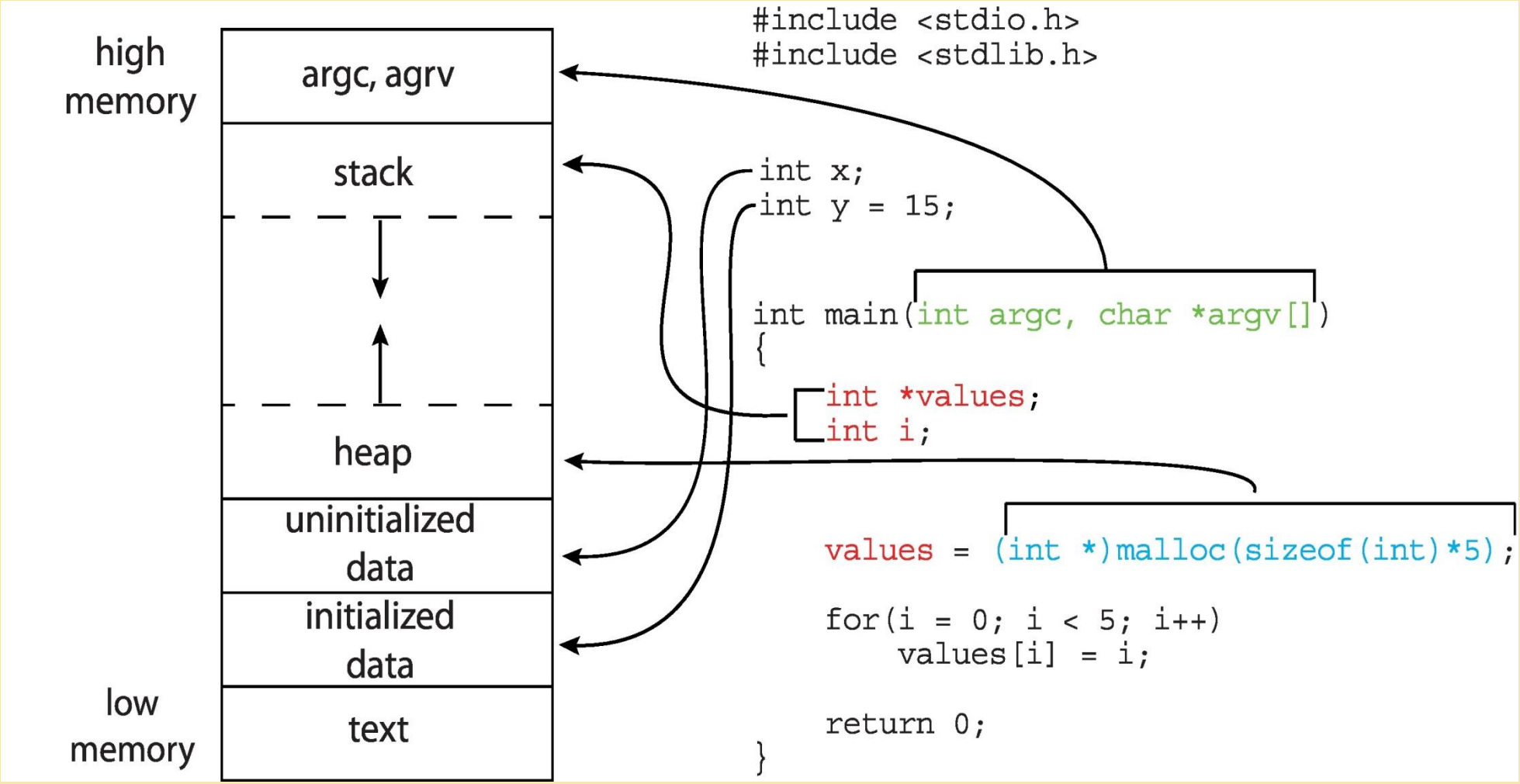
Khái niệm tiến trình

- Chương trình là thực thể thụ động (passive) được lưu trữ trên đĩa (Executable file - tập tin thực thi), tiến trình là thực thể hoạt động (active)
  - **Chương trình** trở thành **tiến trình** khi tập tin thực thi được đưa vào trong bộ nhớ
- Việc thực thi chương trình được bắt đầu bằng: đúp chuột trong giao diện đồ họa; gõ tên lệnh trong giao diện dòng lệnh
- Một chương trình có thể có nhiều tiến trình
  - Nhiều người dùng cùng chạy một loại chương trình

Tiến trình trong bộ nhớ



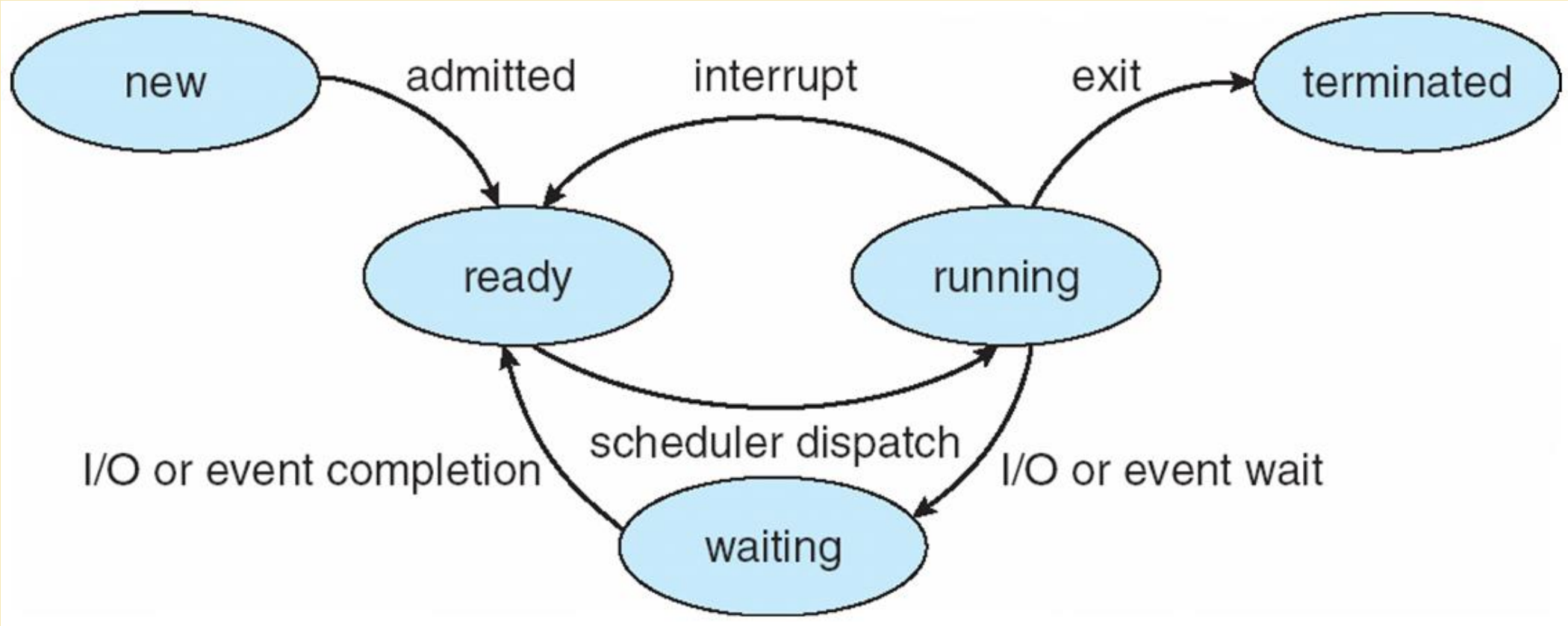
Bố cục trong bộ nhớ của một chương trình C



Trạng thái tiến trình

- Khi một tiến trình thực thi, nó có thể thay đổi trạng thái
  - **New**: tiến trình đang được tạo
  - **Running**: tập lệnh đang được thực thi
  - **Waiting**: tiến trình đang chờ một số sự kiện xảy ra
  - **Ready**: tiến trình đang chờ nhận bộ xử lý (CPU)
  - **Terminated**: tiến trình hoàn thành việc thực thi

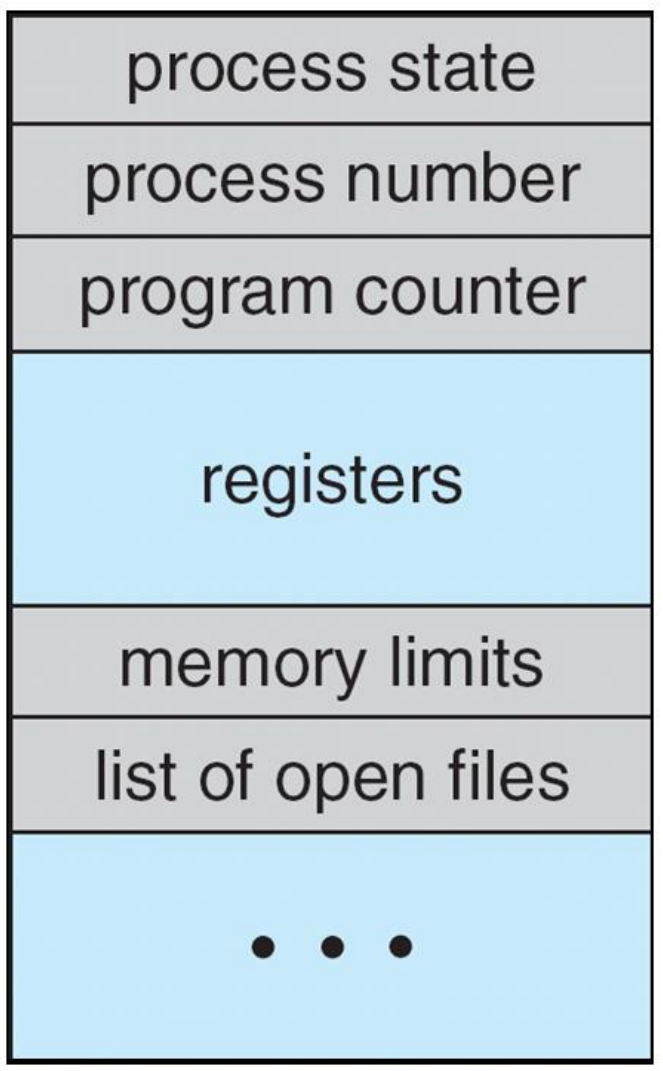
Sơ đồ trạng thái tiến trình



Khối điều khiển tiến trình (Process Control Block - PCB)

Thông tin được liên kết vào mỗi tiến trình (Còn gọi là khối điều khiển tác vụ)

- Trạng thái tiến trình – running, waiting,…
- Bộ đếm chương trình – vị trí của lệnh thực thi tiếp theo
- Thanh ghi – nội dung của tất cả tiến trình
- Thông tin định thời CPU – độ ưu tiên, con trỏ hàng đợi
- Thông tin quản lý bộ nhớ - bộ nhớ được cấp phát cho tiến trình
- Thông tin thống kê - CPU được sử dụng, đồng hồ đếm thời gian, giới hạn thời gian
- Thông tin trạng thái Nhập/Xuất – các thiết bị được cấp phát cho tiến trình, danh sách các tập tin



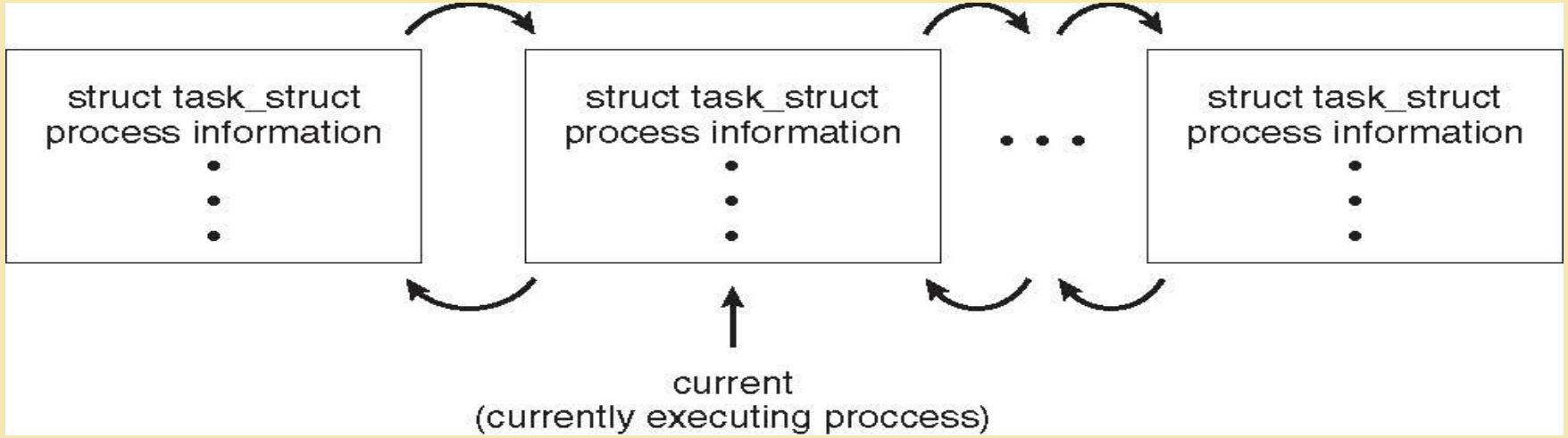
Tiểu trình

- Đến thời điểm đang xét, tiến trình có 1 tiểu trình (Thread) duy nhất thực thi
- Xét trường hợp có nhiều bộ đếm chương trình trên tiến trình
  - Nhiều vị trí lệnh có thể thực thi cùng một lúc
    - Nhiều tiểu trình điều khiển → khảo sát ở chương **Tiểu trình (Threads)**
- Phải có không gian lưu trữ chi tiết cho các tiểu trình, nhiều bộ đếm chương trình trong PCB

Sự biểu diễn tiến trình trong Linux

Được biểu diễn bằng cấu trúc ngôn ngữ C ***stack_struck***

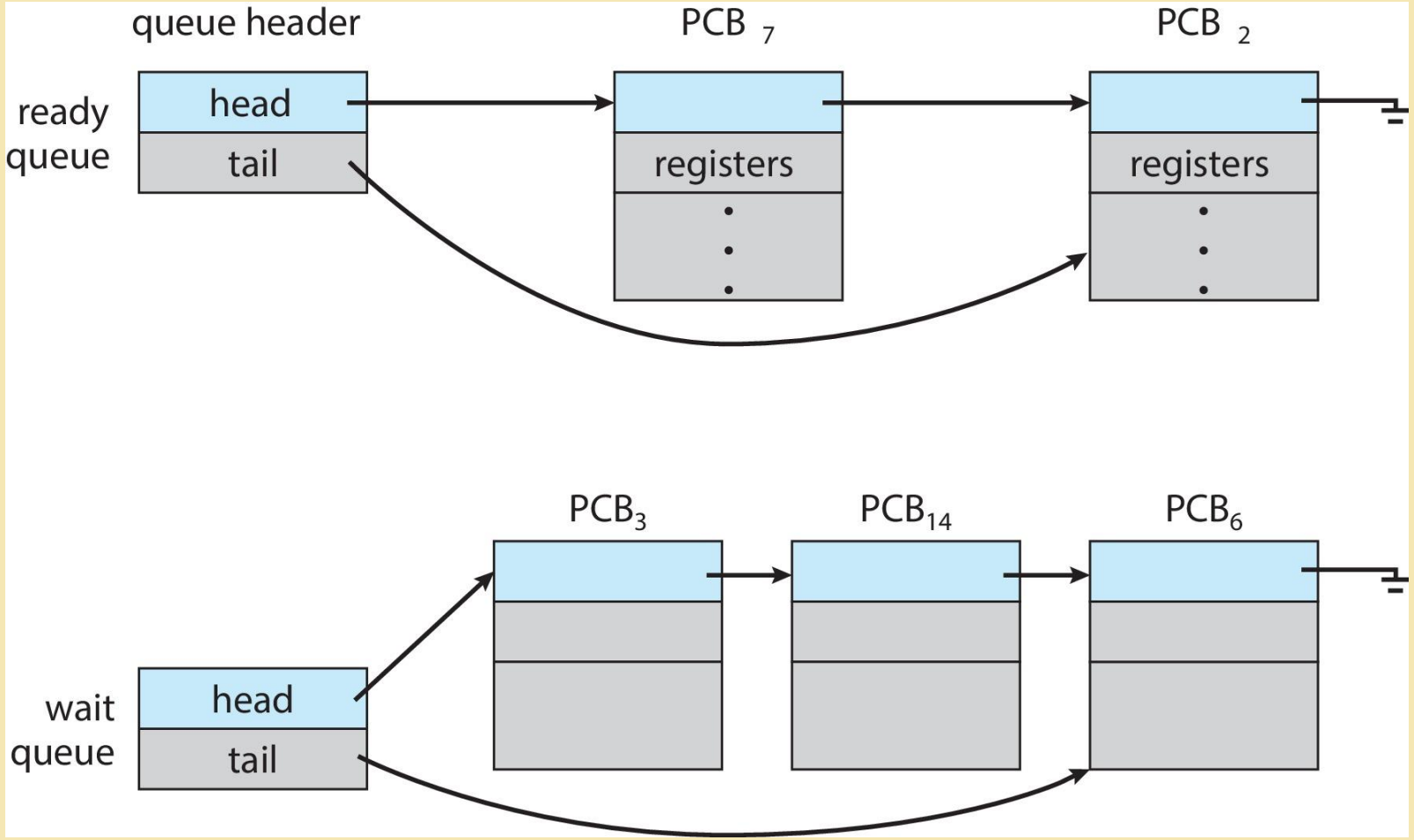
```
pid t_pid; /* process identifier */
long state; /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this process */
```



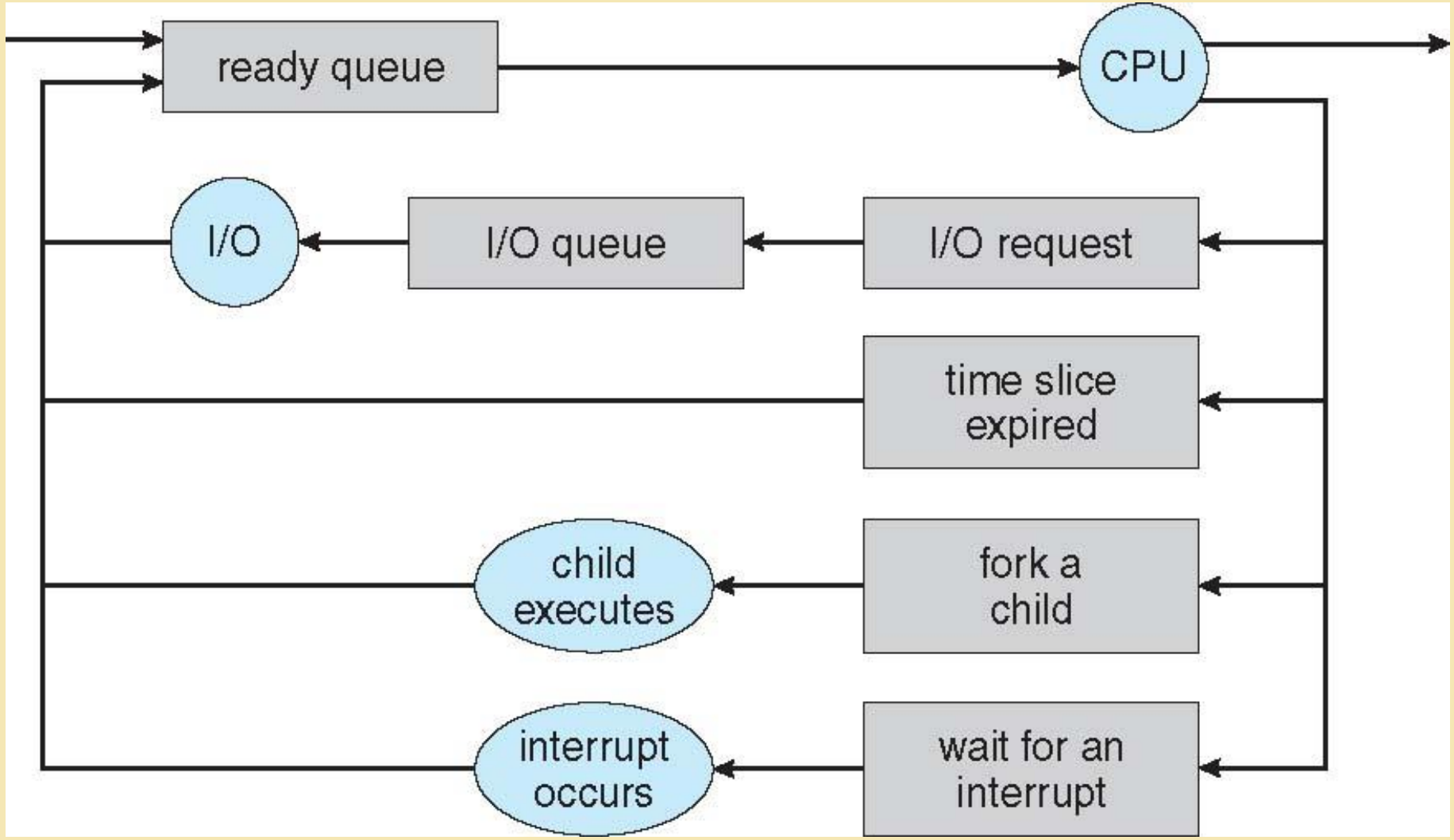
2. Định thời tiến trình

- Tối đa việc sử dụng CPU, chuyển tiếp tức thì tiến trình vào CPU core
- Bộ định thời tiến trình lựa chọn 1 trong các tiến trình có sẵn để thực thi tiếp theo với CPU core
- Duy trì hàng đợi định thời của các tiến trình
  - **Ready queue** – tập tất cả các tiến trình đang có trong bộ nhớ chính, sẵn sàng và chờ thực thi
  - **Wait queue** – tập các tiến trình đang chờ một sự kiện (nhập/xuất, …)
  - Các tiến trình di chuyển giữa các hàng đợi

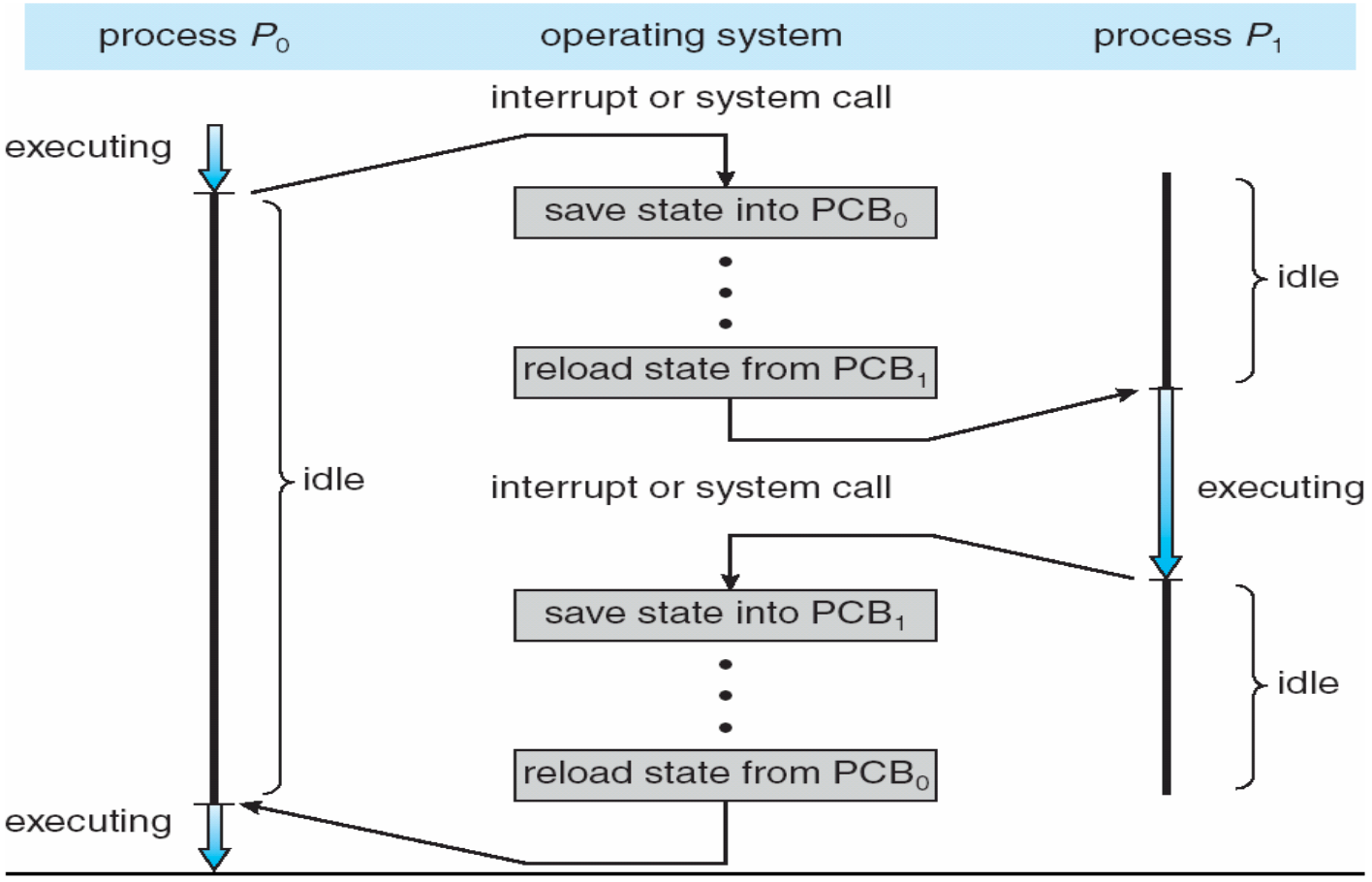
Ready queue và Wait queue



Biểu diễn định thời tiến trình



Chuyển CPU từ tiến trình tới tiến trình



Chuyển ngữ cảnh xảy ra khi CPU chuyển tiếp từ một tiến trình tới tiến trình khác

Chuyển ngữ cảnh

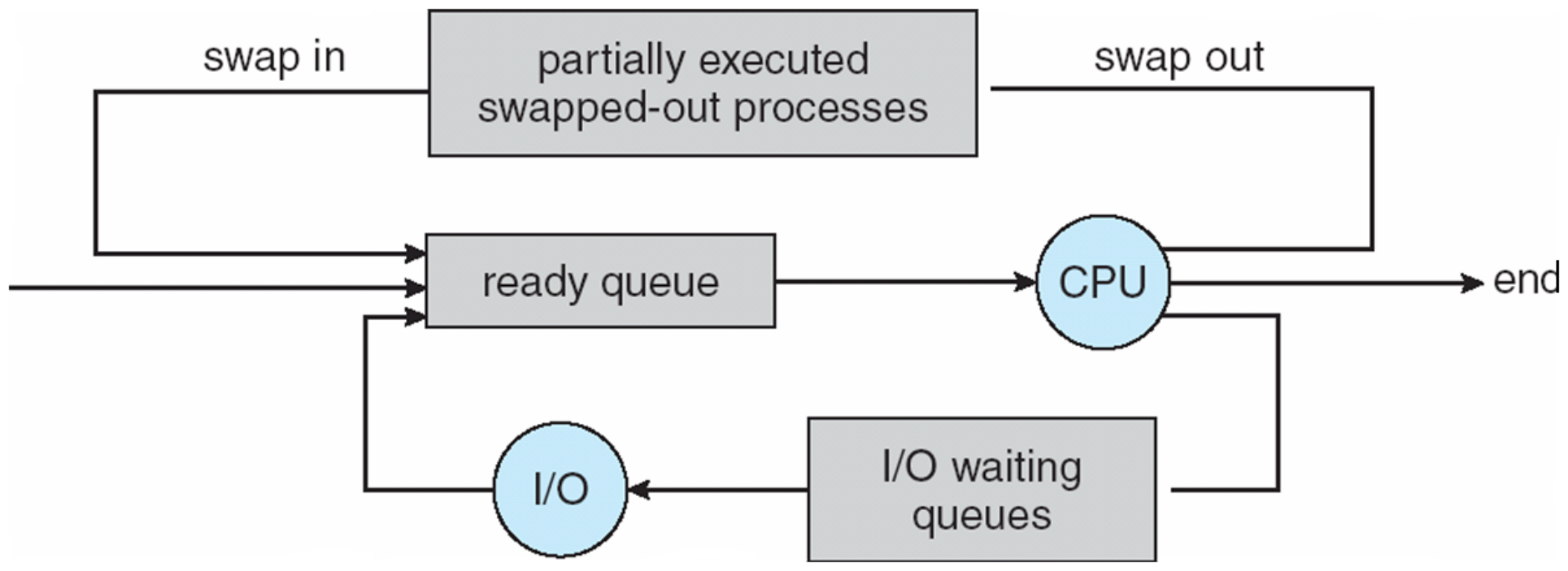
- Khi CPU chuyển sang một tiến trình khác, hệ thống phải lưu lại trạng thái của tiến trình cũ và tải trạng thái đã lưu cho tiến trình mới bằng cách chuyển ngữ cảnh (**context switch**)
- Ngữ cảnh của tiến trình được thể hiện trong PCB
- Thời gian chuyển ngữ cảnh là chi phí quản lý đầu tiên; hệ thống không có công việc hữu ích trong khi chuyển
  - HĐH và PCB càng phức tạp → chuyển ngữ cảnh càng lâu
- Thời gian phụ thuộc vào phần cứng hỗ trợ
  - Một số phần cứng cung cấp nhiều tập thanh ghi trên CPU → nhiều ngữ cảnh được tải cùng lúc

Các bộ định thời

- Bộ định thời ngắn hạn (Short-term Scheduler hoặc CPU Scheduler) – chọn tiến trình sẽ được thực thi tiếp theo và cấp phát CPU
  - Bộ định thời ngắn hạn được gọi thường xuyên (mili giây) → phải nhanh chóng
- Bộ định thời dài hạn (Long-term Scheduler hoặc job Scheduler) – chọn tiến trình được đưa vào ready queue
  - Bộ định thời dài hạn được gọi không thường xuyên (giây, phút) → có thể chậm
  - Bộ định thời dài hạn kiểm soát mức độ đa chương (số tiến trình trong bộ nhớ)
- Tiến trình có thể được mô tả theo 2 loại:
  - Tiến trình thiên hướng vào ra (I/O-bound process) – tốn nhiều thời gian cho việc nhập xuất hơn việc tính toán, có nhiều phiên sử dụng CPU ngắn
  - Tiến trình thiên hướng xử lý (CPU-bound process) – tốn nhiều thời gian cho việc tính toán, có vài phiên sử dụng CPU dài
- Bộ định thời dài hạn cố gắng làm cho 2 loại tiến trình kết hợp làm việc hiệu quả nhất

Các bộ định thời

- Bộ định thời trung hạn (Medium-term Scheduler) – được bổ sung khi mức độ đa chương cần giảm
  - Di chuyển tiến trình từ bộ nhớ, lưu trữ vào đĩa, lấy lại từ đĩa để thực thi tiếp tục: **swapping**



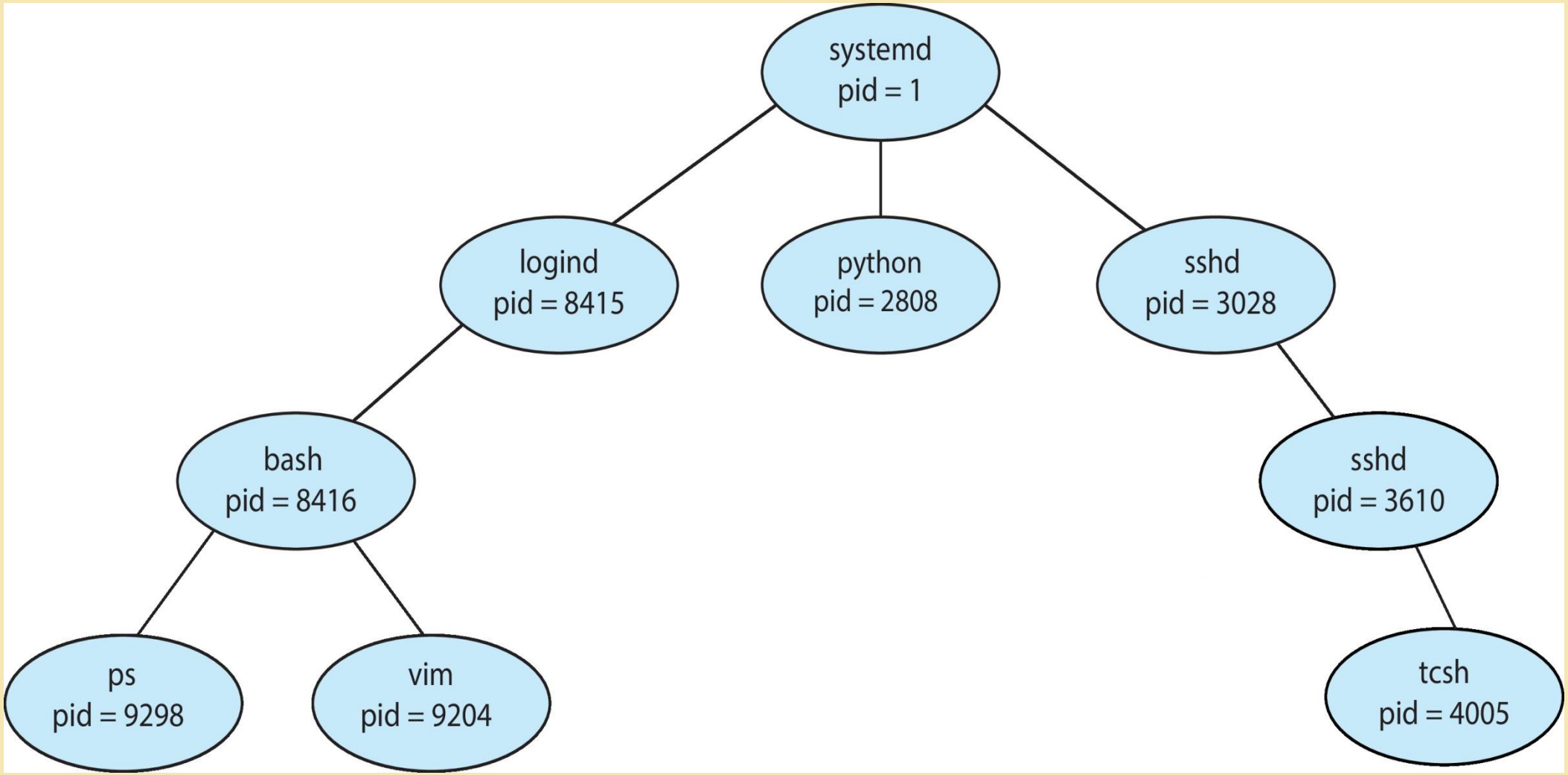
3. Các thao tác trên tiến trình

- Hệ thống phải cung cấp các cơ chế:
  - Tạo tiến trình
  - Kết thúc tiến trình

Tạo tiến trình

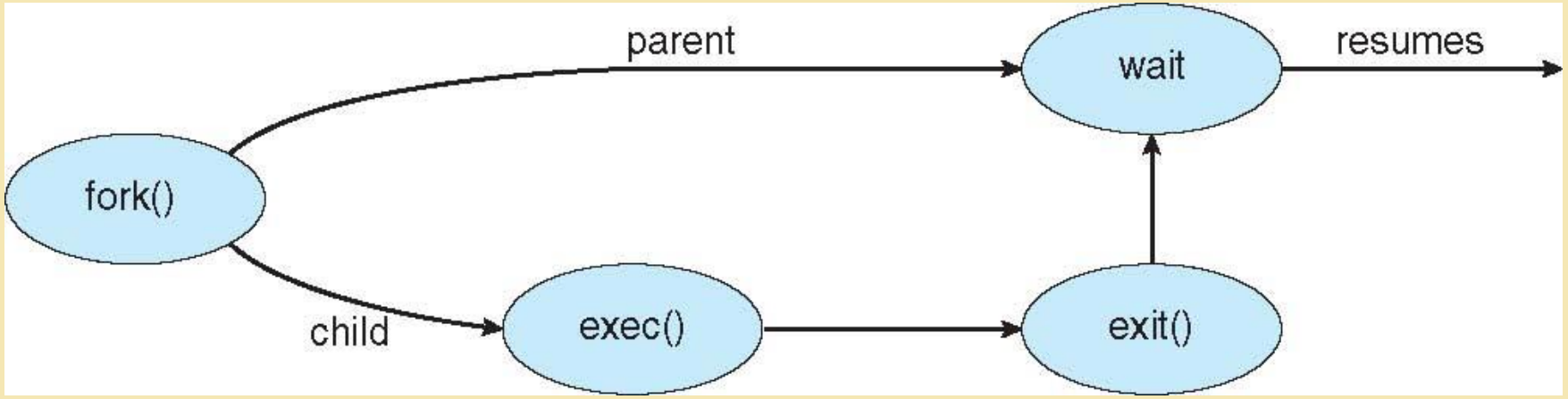
- Tiến trình cha tạo tiến trình con, tiến trình con lại tạo ra các tiến trình khác, hình thành cây tiến trình
- Tiến trình được xác định và quản lý thông qua mã định danh tiến trình (process identifier – **pid**)
- Các tùy chọn chia sẻ tài nguyên
  - Tiến trình cha và tiến trình con chia sẻ tất cả tài nguyên
  - Tiến trình con sử dụng chung tập tài nguyên con của tiến trình cha
  - Tiến trình cha và tiến trình con không chia sẻ tài nguyên
- Các tùy chọn thực thi
  - Tiến trình cha và tiến trình con thực thi đồng thời
  - Tiến trình cha chờ đến khi tiến trình con kết thúc

Cây tiến trình trong Linux



Tạo tiến trình

- Không gian địa chỉ
  - Tiến trình con là một bản sao của tiến trình cha (nó có cùng chương trình và dữ liệu như tiến trình cha)
  - Tiến trình con là một chương trình mới được nạp vào
- Ví dụ trên UNIX
  - Lời gọi hệ thống ***fork()*** tạo ra một tiến trình mới
  - Lời gọi hệ thống ***exec()*** được sử dụng sau lời gọi ***fork()*** để thay thế không gian bộ nhớ của tiến trình bằng một chương trình mới



Chương trình C: tạo 1 tiến trình riêng biệt sử dụng lời gọi hệ thống fork() trong UNIX

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

Tạo tiến trình riêng biệt bằng Windows API

```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
STARTUPINFO si;
PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
     "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
     NULL, /* don't inherit process handle */
     NULL, /* don't inherit thread handle */
     FALSE, /* disable handle inheritance */
     0, /* no creation flags */
     NULL, /* use parent's environment block */
     NULL, /* use parent's existing directory */
     &si,
     &pi))
    {
       fprintf(stderr, "Create Process Failed");
       return -1;
    }
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```

Kết thúc tiến trình

- Tiến trình thực thi câu lệnh cuối cùng và sau đó yêu cầu hệ điều hành xóa nó bằng cách sử dụng lời gọi hệ thống ***exit()***
  - Trạng thái dữ liệu từ tiến trình con được trả về cho tiến trình cha (thông qua lời gọi hệ thống ***wait()***)
  - Các tài nguyên của tiến trình được giải phóng bởi HĐH
- Tiến trình cha kết thúc việc thực thi của tiến trình con bằng lời gọi hệ thống ***abort()*** vì một số lý do:
  - Tiến trình con sử dụng vượt mức tài nguyên nó được cấp phát
  - Tác vụ được gán cho tiến trình con không còn cần nữa
  - Tiến trình cha kết thúc và HĐH không cho phép tiến trình con tiếp tục nếu tiến trình cha kết thúc

Kết thúc tiến trình

- Một số hệ thống không cho phép một tiến trình con tồn tại nếu tiến trình cha của nó đã chấm dứt. Nếu một tiến trình kết thúc (bình thường hoặc bất thường), thì sau đó tất cả các tiến trình con của nó cũng phải được kết thúc.
  - Kết thúc theo tầng (cascading termination). Tất cả tiến trình con, cháu,… đều kết thúc
  - Việc kết thúc được thực hiện bởi HĐH
- Các tiến trình cha có thể chờ đợi sự kết thúc của tiến trình con bằng cách dùng lời gọi hệ thống ***wait()***. Lời gọi này trả về trạng thái thông tin và mã pid của tiến trình kết thúc

```
pid = wait(&status);
```

- Một tiến trình đã chấm dứt, nhưng tiến trình cha vẫn chưa gọi lời gọi ***wait()***, thì được gọi là tiến trình **zombie**
- Nếu tiến trình cha kết thúc mà không gọi lời gọi ***wait()***, thì tiến trình con được gọi là tiến trình **orphan**

Kiến trúc đa tiến trình – Chrome Browser

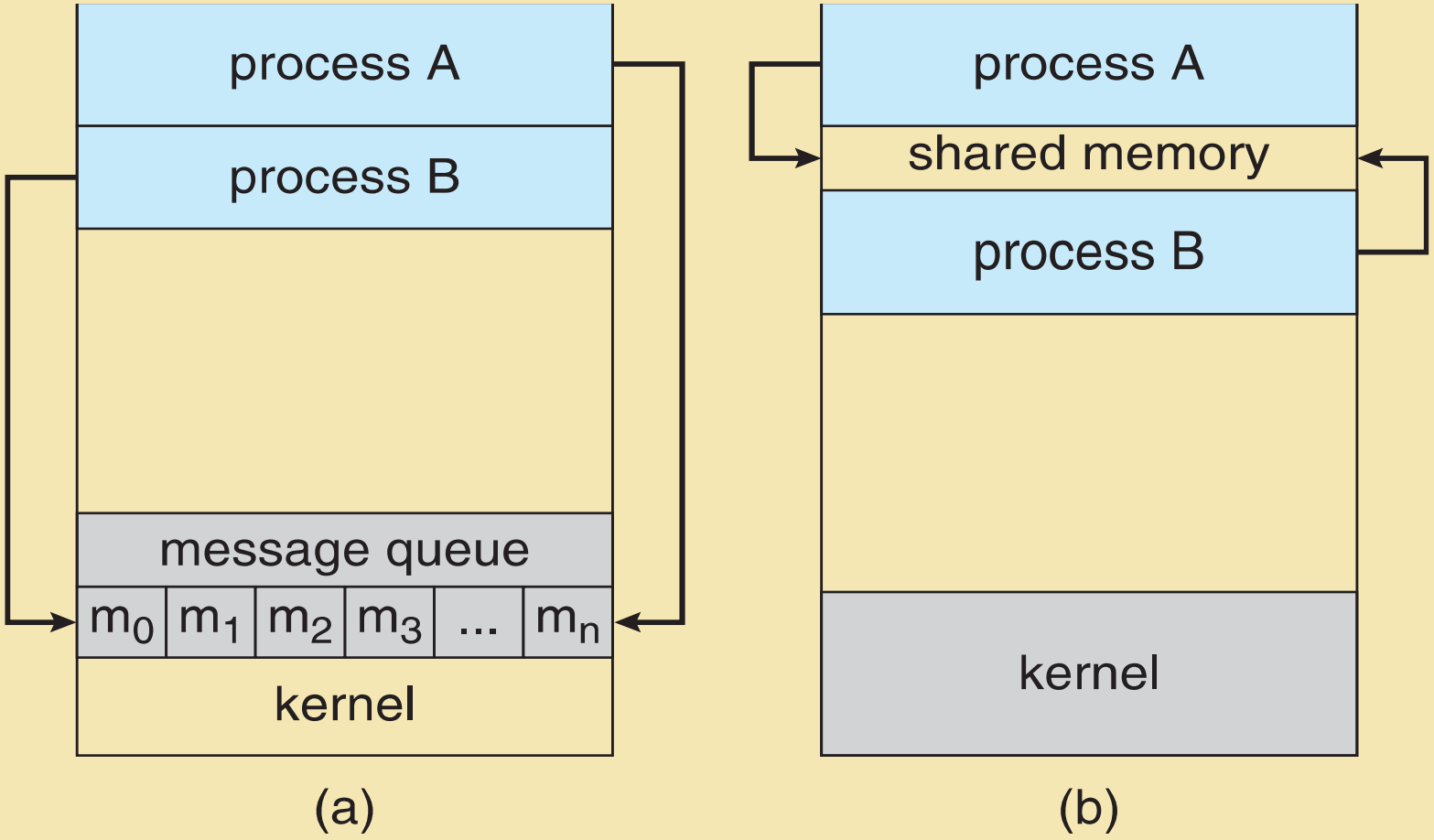
- Nhiều trình duyệt web chạy như một đơn tiến trình
  - Nếu một trang web có lỗi, toàn bộ trình duyệt có thể bị treo hoặc sụp đổ
- Trình duyệt Google Chrome là đa tiến trình với 3 loại tiến trình khác nhau
  - Tiến trình Browser: quản lý giao diện, đĩa và nhập/xuất
  - Tiến trình Renderer: trình diễn các trang web, HTML, Javascript. Một tiến trình Renderer được tạo mỗi khi website được mở
    - Chạy trong sandbox thì hạn chế truy cập đĩa và I/O, giảm thiểu tác động của các khai thác bảo mật
  - Tiến trình plug-in đối với từng loại plug-in

4. Truyền thông liên tiến trình

- Tiến trình trong một hệ thống có thể là tiến trình độc lập hoặc tiến trình cộng tác
- Tiến trình cộng tác có thể ảnh hưởng hoặc bị ảnh hưởng bởi tiến trình khác, bao gồm dữ liệu chia sẻ
- Có nhiều lý do để các tiến trình cộng tác
  - Chia sẻ thông tin
  - Tăng tốc độ tính toán
  - Đơn thể hóa
  - Sự thuận tiện
- Tiến trình cộng tác cần cơ chế truyền thông liên tiến trình (interprocess communication - IPC)
- Hai mô hình IPC
  - Bộ nhớ chia sẻ (Shared memory)
  - Trao đổi thông điệp (Message passing)

Mô hình truyền thông

(a) Message passing. (b) shared memory.



Tiến trình cộng tác

- Tiến trình độc lập không thể ảnh hưởng hoặc bị ảnh hưởng bởi việc thực thi của các tiến trình khác
- Tiến trình cộng tác có thể ảnh hưởng hoặc bị ảnh hưởng bởi việc thực thi của các tiến trình khác
- Lợi ích của việc cộng tác
  - Chia sẻ thông tin
  - Tăng tốc độ tính toán
  - Đơn thể hóa
  - Sự thuận tiện

Bài toán Producer-Consumer

- Mô hình tiến trình cộng tác: tiến trình Producer sản xuất thông tin được tiêu thụ bởi tiến trình Consumer
  - Vùng đệm không giới hạn (Unbounded-buffer) – không giới hạn kích thước vùng đệm
  - Vùng đệm giới hạn (Bounded-buffer) – kích thước vùng đệm được gán cố định

Bounded-buffer – giải pháp bộ nhớ chia sẻ

- Dữ liệu chia sẻ

```
#define BUFFER_SIZE 10
typedef struct {
  . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Giải pháp đúng, nhưng chỉ có thể sử dụng ***BUFFER_SIZE - 1*** phần tử

Bounded-buffer – tiến trình Producer

```
item next_produced;
while (true) {
        /* produce an item in next produced */
        while (((in + 1) % BUFFER_SIZE) == out)
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
}
```

Bounded-buffer – tiến trình Consumer

```
item next_consumed;
while (true) {
        while (in == out)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        /* consume the item in next consumed */
}
```

Truyền thông liên tiến trình – giải pháp bộ nhớ chia sẻ

- Một vùng của bộ nhớ được chia sẻ giữa các tiến trình có nhu cầu liên lạc
- Liên lạc dưới sự kiểm soát của các tiến trình người sử dụng, không phải hệ điều hành
- Vấn đề chính là cung cấp cơ chế cho phép các tiến trình người dùng đồng bộ hóa các hoạt động của chúng khi chúng truy cập bộ nhớ chia sẻ
- Đồng bộ hóa sẽ được thảo luận trong chương "ĐỒNG BỘ HÓA TIẾN TRÌNH"

Truyền thông liên tiến trình – giải pháp trao đổi thông điệp

- Cơ chế cho tiến trình truyền thông và đồng bộ các hoạt động của chúng
- Thông điệp hệ thống - các tiến trình liên lạc với nhau mà không cần đến các biến chia sẻ
- IPC cung cấp hai hoạt động
  - **Send**(*message*)
  - **Receive**(*message*)
- Kích thước *message* là cố định hoặc biến đổi

Truyền thông liên tiến trình – giải pháp trao đổi thông điệp

- Nếu tiến trình P và Q cần liên lạc với nhau, chúng cần
  - Thiết lập một liên kết truyền thông giữa chúng
  - Trao đổi thông điệp qua việc gửi/nhận (*send/receive*)
- Vấn đề thực hiện:
  - Thiết lập liên kết như thế nào?
  - Một liên kết có thể được liên kết với hơn hai tiến trình không?
  - Bao nhiêu liên kết có thể có giữa mỗi cặp tiến trình?
  - Năng suất của một liên kết?
  - Kích thước của thông điệp mà liên kết chứa đựng là cố định hay biến đổi?
  - Liên kết một chiều hay hai chiều?

Truyền thông liên tiến trình – giải pháp trao đổi thông điệp

- Thực hiện liên kết truyền thông
  - Về mặt vật lý
    - Bộ nhớ chia sẻ
    - Đường truyền
    - Mạng
  - Về mặt logic
    - Trực tiếp hoặc gián tiếp
    - Đồng bộ hoặc không đồng bộ
    - Vùng đệm tự động hoặc chi tiết

Truyền thông trực tiếp

- Tiến trình phải được đặt tên một cách rõ rang
  - Send(P,message) – gởi thông điệp tới tiến trình P
  - Receive(Q,message) – nhận thông điệp từ tiến trình Q
- Thuộc tính của liên kết
  - Liên kết được thiết lập tự động
  - Một liên kết được gắn chính xác với một cặp tiến trình truyền thông
  - Giữa mỗi cặp tồn tại đúng một liên kết
  - Các liên kết có thể là một chiều, nhưng thường là hai chiều

Truyền thông gián tiếp

- Thông điệp được gởi và nhận từ hộp thư (mailboxs) (cũng dựa vào cổng - ports)
  - Mỗi hộp thư có một mã ***id*** duy nhất
  - Tiến trình có thể liên lạc chỉ khi nó chia sẻ hộp thư
- Thuộc tính của liên kết
  - Liên kết chỉ được thiết lập nếu các tiến trình chia sẻ một hộp thư chung
  - Một liên kết có thể được gắn với nhiều tiến trình
  - Mỗi cặp tiến trình có thể chia sẻ một số liên kết
  - Liên kết có thể là một chiều hoặc hai chiều

Truyền thông gián tiếp

- Các thao tác
  - Tạo hộp thư mới (cổng)
  - Gởi và nhận thông điệp thông qua hộp thư
  - Hủy hộp thư
- Các hàm nguyên thủy được định nghĩa như sau:
  - **Send**(A,message) – gởi một thông điệp tới hộp thư A
  - **Receive**(A,message) – nhận một thông điệp từ hộp thư A

Truyền thông gián tiếp

- Hộp thư chia sẻ
  - P1,P2 và P3 chia sẻ hộp thư A
  - P1 gởi thông điệp, P2 và P3 nhận thông điệp
  - Tiến trình nào nhận thông điệp?
- Giải pháp
  - Cho phép một liên kết được gắn với nhiều nhất hai tiến trình
  - Tại một thời điểm, chỉ cho phép duy nhất một tiến trình nhận
  - Cho phép hệ thống lựa chọn ngẫu nhiên tiến trình nhận. Tiến trình gởi thông báo tên của tiến trình nhận

Đồng bộ hóa

- Cơ chế trao đổi thông điệp có thể thực hiện theo 2 cách: Blocking hoặc non-blocking
- Blocking được xem như đồng bộ hóa
  - Blocking send – tiến trình gởi bị khóa cho đến khi thông điệp được nhận
  - Blocking receive – tiến trình nhận bị khóa cho đến khi thông điệp gởi tới
- Non-blocking được xem như không đồng bộ
  - Non-blocking send – tiến trình gởi gởi thông điệp đi và tiếp tục
  - Non-blocking receive – tiến trình nhận nhận thông điệp:
    - Hợp lệ
    - Không hợp lệ (rỗng – null)

Đồng bộ hóa

- Bài toán Producer-Consumer trở thành bình thường

```
message next_produced;
        while (true) {
            /* produce an item in next produced */
        send(next_produced);
        }
 message next_consumed;
 while (true) {
    receive(next_consumed);

    /* consume the item in next consumed */
 }
```

Vùng đệm

- Hàng đợi thông điệp được gán vào liên kết
- Thực thi bằng 1 trong 3 cách
  - Zero capacity – không có thông điệp trong liên kết. Tiến trình gởi phải chờ tiến trình nhận
  - Bounded capacity – giới hạn độ dài của n thông điệp. Tiến trình gởi phải chờ nếu liên kết đầy
  - Unbounded capacity – không giới hạn độ dài. Tiến trình gởi không bao giờ chờ

Thắc mắc & Giải đáp